

GenAI Interview Questions & Answers

Most important questions for GenAI / LLM / RAG internships — brief interview-ready answers.

1. What is Generative AI?

Answer: Generative AI is a type of AI that learns patterns from existing data and generates new content such as text, code, images, audio, or video. LLMs are a major example for text generation.

2. How is Generative AI different from traditional Machine Learning?

Answer: Traditional ML mainly predicts or classifies outcomes from input data, while Generative AI creates new content. Example: spam detection is traditional ML; generating an email is Generative AI.

3. What is an LLM?

Answer: A Large Language Model is a neural network trained on a very large amount of text to understand and generate human-like language. Examples include GPT, Llama, and Mistral.

4. How does an LLM generate text?

Answer: The model tokenizes the input, converts tokens into representations, processes them through Transformer layers, predicts probabilities for the next token, selects a token using a decoding strategy, and repeats this process until the response is complete.

5. What is a Transformer?

Answer: A Transformer is a neural-network architecture built around attention mechanisms. It processes relationships between tokens efficiently and is the foundation of most modern LLMs.

6. Why are Transformers preferred over RNNs for LLMs?

Answer: Transformers can process many tokens in parallel during training and capture long-range relationships more effectively. RNNs process sequences step-by-step, which makes them harder to scale.

7. What is self-attention?

Answer: Self-attention lets each token consider other tokens in the same sequence and determine which ones are important for understanding the current token.

8. What are Query, Key and Value?

Answer: Query represents what a token is looking for, Key represents what each token offers for matching, and Value contains the information that is actually combined. Attention scores determine how strongly values contribute.

9. What is tokenization?

Answer: Tokenization converts text into smaller units called tokens. A token may be a word, part of a word, punctuation mark, or special symbol depending on the tokenizer.

10. What is an embedding?

Answer: An embedding is a numerical vector representation of text that captures semantic meaning. Similar meanings tend to have vectors that are close in vector space.

11. GPT vs BERT vs T5?

Answer: GPT is mainly decoder-based and designed for text generation. BERT is encoder-based and designed mainly for understanding tasks. T5 uses an encoder-decoder design and treats many NLP tasks as text-to-text problems.

12. What is pre-training?

Answer: Pre-training is the initial training stage where a model learns general language patterns from a very large dataset, usually using a self-supervised objective.

13. What is fine-tuning?

Answer: Fine-tuning further trains a pre-trained model on a smaller, task-specific dataset so it performs better for a particular domain or task.

14. What is instruction tuning?

Answer: Instruction tuning trains a model on examples of instructions and desired responses, making the model better at following user commands.

15. What is RLHF?

Answer: Reinforcement Learning from Human Feedback uses human preferences to train or optimize a model toward responses that people consider more useful, safe, or aligned.

16. What is RAG?

Answer: Retrieval-Augmented Generation combines information retrieval with an LLM. Relevant external documents are retrieved and provided as context so the model can answer using that information.

17. Why do we need RAG?

Answer: RAG helps answer questions using private or frequently changing data without retraining the model. It can also reduce hallucinations when retrieval is accurate.

18. RAG vs Fine-tuning?

Answer: RAG changes the information supplied to the model at inference time; fine-tuning changes the model's learned behavior. RAG is usually better for dynamic knowledge, while fine-tuning is useful for behavior, style, or specialized task patterns.

19. What is a context window?

Answer: The context window is the maximum amount of tokens a model can process as input/context for a request. If the supplied content is too large, some content must be removed, summarized, or otherwise reduced.

20. What is hallucination?

Answer: Hallucination is when an LLM produces information that sounds convincing but is incorrect, unsupported, or fabricated.

21. How can hallucinations be reduced?

Answer: Use reliable retrieval, clear prompts, grounding instructions, lower randomness when appropriate, structured outputs, validation, and a fallback such as 'I don't know' when evidence is insufficient.

22. What is temperature?

Answer: Temperature controls randomness during token selection. Lower values make output more deterministic; higher values generally make it more diverse and unpredictable.

23. What are top-k and top-p?

Answer: Top-k limits selection to the k most probable tokens. Top-p selects from the smallest group of tokens whose cumulative probability reaches p. Both control sampling diversity.

24. What is inference?

Answer: Inference is the process of using a trained model to produce predictions or generated output for new input.

25. What is quantization?

Answer: Quantization reduces the numerical precision used to store or compute model weights, such as from 16-bit to 8-bit or 4-bit. It can reduce memory usage and often improve inference efficiency.

26. What do 7B, 13B and 70B mean?

Answer: They indicate approximately how many parameters the model has: 7 billion, 13 billion, and 70 billion. More parameters can increase capability but usually require more compute and memory.

27. Explain the RAG pipeline.

Answer: Typical pipeline: Load documents → split into chunks → create embeddings → store vectors → retrieve relevant chunks for a query → optionally rerank → send query plus retrieved context to the LLM → generate a grounded answer.

28. Why split documents into chunks?

Answer: Chunking breaks large documents into manageable pieces so retrieval can return focused, relevant context instead of an entire document.

29. What are chunk size and overlap?

Answer: Chunk size controls how much text is placed in each chunk. Chunk overlap repeats a small amount of text between neighboring chunks to preserve context across boundaries.

30. What is a vector database?

Answer: A vector database stores embeddings and supports efficient similarity search. Examples include FAISS, Chroma, Pinecone, and Weaviate.

31. What is semantic search?

Answer: Semantic search finds content based on meaning rather than exact keyword matches. It commonly uses embeddings and vector similarity.

32. What is cosine similarity?

Answer: Cosine similarity measures how similar two vectors are by comparing the angle between them. It is commonly used to compare text embeddings.

33. Keyword search vs vector search?

Answer: Keyword search mainly matches words or terms. Vector search compares semantic representations, so it can find relevant content even when the exact words differ.

34. What is hybrid search?

Answer: Hybrid search combines keyword-based retrieval and vector-based semantic retrieval to improve recall and handle both exact terms and semantic meaning.

35. What is reranking?

Answer: Reranking takes retrieved candidates and uses a stronger ranking model to reorder them by relevance before sending the best context to the LLM.

36. How do you improve RAG accuracy?

Answer: Improve chunking, use a strong embedding model, tune retrieval parameters, add metadata filters, use hybrid retrieval or reranking, improve prompts, and evaluate retrieval separately from generation.

37. What is LangChain?

Answer: LangChain is a framework for building LLM applications using components such as prompts, models, document loaders, retrievers, tools, chains, and agents.

38. What is a retriever in LangChain?

Answer: A retriever receives a query and returns relevant documents or document chunks from a data source such as a vector store.

39. What is a document loader?

Answer: A document loader imports data from sources such as PDFs, text files, webpages, or other supported formats into a form that can be processed by the application.

40. What is a text splitter?

Answer: A text splitter divides documents into smaller chunks before embedding and retrieval. RecursiveCharacterTextSplitter is commonly used because it tries to preserve natural text boundaries.

41. What is LangGraph?

Answer: LangGraph is a framework for building stateful, multi-step, and agentic workflows as graphs. It is useful when an application needs controlled steps, branching, loops, tools, or persistent state.

42. What is Hugging Face?

Answer: Hugging Face is an AI platform and ecosystem providing models, datasets, libraries, and tools for machine learning and generative AI.

43. What is HuggingFaceEndpoint?

Answer: In LangChain, HuggingFaceEndpoint is used to interact with models hosted through Hugging Face inference services/endpoints rather than necessarily running the model locally.

44. What is prompt engineering?

Answer: Prompt engineering is the practice of designing instructions and context so an LLM produces more accurate, consistent, and useful outputs.

45. Zero-shot vs few-shot prompting?

Answer: Zero-shot gives only the task instruction. Few-shot includes a small number of examples showing the desired input-output pattern.

46. What is structured output?

Answer: Structured output forces or guides the model to return data in a predictable schema, such as JSON with specific fields. This makes LLM output easier for software to validate and consume.

47. What is prompt injection?

Answer: Prompt injection is an attack where untrusted input attempts to manipulate the model into ignoring system instructions or performing unintended actions.

48. What is an AI agent?

Answer: An AI agent is an LLM-based system that can reason over a task, choose actions or tools, observe results, and continue until it reaches a goal.

49. Agent vs chatbot?

Answer: A chatbot mainly responds to conversations. An agent can additionally plan or decide actions, call tools, use external data, and execute multi-step workflows.

50. What is tool/function calling?

Answer: Tool calling lets an LLM request a predefined function with structured arguments. The application executes the function and returns the result to the model.

51. What is the ReAct pattern?

Answer: ReAct combines reasoning with actions: the model determines what it needs to do, calls a tool, observes the result, and continues the workflow.

52. How would you build a PDF QA application?

Answer: Load PDFs with a document loader → split text → create embeddings → store vectors → retrieve relevant chunks for a question → pass retrieved context to an LLM → return the grounded answer.

53. Why choose RAG for a project?

Answer: Choose RAG when the application must answer from external, private, or changing knowledge. It avoids retraining the model whenever the source documents change.

54. How would you reduce LLM cost?

Answer: Use smaller models when sufficient, cache repeated results, reduce unnecessary context, optimize prompts, route simple tasks to cheaper models, batch work where possible, and monitor token usage.

55. How would you reduce latency?

Answer: Reduce prompt/context size, retrieve fewer but better chunks, use faster models, stream responses, cache results, optimize vector search, and avoid unnecessary sequential tool calls.

56. How do you secure API keys?

Answer: Keep secrets in environment variables or a secret manager, never commit them to Git, restrict permissions, rotate leaked keys, and keep keys on the server rather than exposing them in frontend code.

57. How do you evaluate a RAG application?

Answer: Evaluate retrieval relevance and answer quality separately. Useful measures include retrieval precision/recall-style metrics, groundedness, correctness, relevance, latency, cost, and human evaluation.

58. What happens when no relevant document is found?

Answer: The system should avoid inventing an answer. It can return a controlled fallback such as 'I couldn't find enough information in the provided documents.'